Team Members [Team 4.5]: Minghao Zou, Fumiharu Okutsu, JunHui Guo

Ball Catching in General Room Geometries via Learning Ball Trajectory

Course content covered: Differential IK, Jacobians, motion planning under uncertainty, contact dynamics, point-cloud perception, Drake simulation

Our project is a robot arm learning to catch a ball that bounces off multiple walls in various room geometries, ultimately generalizing to unseen rooms geometries. We will accomplish this in two components: (1) training a neural network to predict a future distribution of trajectories, and (2) using the predicted trajectories to inform a robot action planner that catches the ball at an appropriate future time.

Purpose & Motivation

A promising direction for robotic manipulation in unseen environments is to use world models to predict the future state of the environment given some action, which then informs a planner to make accurate decisions. However, such general world models are often large generative models with billions of parameters, which limits their use on dynamic tasks that require low latency and tight feedback loops. We focus on one such difficult problem: catching a high-velocity ball bouncing in a closed room. Instead of using a general-understanding model, we only need to teach a model the physics of a ball flying and bouncing off objects with different geometries and elasticities.

Accurate modeling of object trajectories is the most important but also difficult challenge, and it is what makes this project interesting to us. Since the task is dynamic, the robot cannot act reactively: it has to learn to predict the ball's motion ahead of time. Moreover, when the ball bounces off multiple walls with unknown elasticity, its future trajectory cannot be predicted with certainty or captured in closed form. We are therefore interested in how accurately a neural network can learn but the trajectories and their uncertainty.

This uncertainty is itself a key focus. A major problem for AI models is overconfidence in their outputs, which is especially harmful for robots interacting with the real world. We will train a model to predict a probability distribution over the ball's future positions instead of a single estimate, since no model can be 100% accurate on this task. This lets us measure how much information about uncertainty helps a robot prepare for multiple possible futures and catch the ball more reliably.

Finally, efficiency is another important component to the task. The robot must keep up with the ball's speed and take in new information as it observes how the ball interacts with the environment. The ball's position at multiple future times must be predicted and updated continuously in real time, alongside the high-level planner and low-level controller. Our algorithms must be fast enough to do all of these computations in real time on a reasonable computer, so we must weigh accuracy against speed. This tradeoff is what motivates a lightweight, specialized model over a large general one in the first place.

Background

Robotic catching and interception. Catching systems usually split the task into three sections: 1) perceiving the ball, 2) predicting its trajectory, and 3) planning an interception. In B. Bauml et al., predictions were refined continuously during flight, and a real-time planner chooses the timing and the configuration of the catch [1]. More recent work plans with inherent uncertainty: Olin et al. uses noisy data to update belief over target states, and start moving the arm while the belief is still being updated [2]. However, these systems assume free ballistic flight of the ball with no bounce in open space. Generalist world models could in principle handle richer environments, but they are too slow: for example, V-JEPA 2, a 1.2B model, needs around 16 seconds to plan one robot action [3], while our regime requires latency on the order of milliseconds.

Learning bounce and trajectory dynamics. In general, the physics of contact is hard to learn for a neural network. Specifically, Parmar et al. show that its nearly discontinuous nature is bad training, generalization, and data efficiency in standard deep networks [4]. Neural Event ODEs [5] handle this by modeling the discontinuity explicitly: they learn event functions which detect instantaneous changes (like a ball bounce) without knowing in advance when they happen. Bounce and Learn [6] instead learns the post-bounce trajectory directly, inferring each surface's restitution and collision normal, and outperforms Newtonian trajectory fitting. However, in our application scenario, predicting the ball motion post-bounce would yield less reaction time and noisier trajectory prediction. Moreover, all of these consider a single surface or a fixed scene, but our setting has several bounces off walls whose geometry and elasticity are unknown.

Representing environment geometry. We need an efficient way of representing and encoding the geometry of a room efficiently to feed to the model. PointNet [7] processes raw, unordered point clouds directly and is invariant to the order of the points, so it can compress a room scan into a global latent vector. Signed distance functions (SDFs) give a local alternative: a SDF returns the distance from a point x to the nearest surface. DeepSDF [8] showed that one latent-conditioned network can represent a whole class of shapes this way.

Our project combines these threads. We train a lightweight, geometry-conditioned, uncertainty-aware trajectory model, and measure how much its predicted distribution improves a real-time catching planner over a single best-guess trajectory.

Proposed solution

Solution stages. We split this project into stages of increasing complexity. Each stage adds one source of difficulty while keeping the rest of the pipeline fixed:

1. **Free flight within reach.** We predict and catch a ball on a ballistic trajectory with no bounces, thrown so that it passes within the arm's workspace. This stage validates the full pipeline (simulation, prediction, planning, and control). An analytic parabola fit serves as a reference, since the problem is nearly solved in closed form.
2. **A single flat wall.** We predict and catch a ball after it bounces off one flat wall at a randomized position, angle, and restitution. This tests whether the model can learn the discontinuous bounce response.
3. **Varied geometries and elasticities.** Bounce surfaces now include angled, curved, and irregular objects with randomized restitution. Restitution cannot be measured beforehand, so the model must infer it from the ball's observed motion. Its predicted uncertainty should shrink as more of the trajectory is seen.ins
4. **General, unseen rooms.** We train on procedurally generated rooms with multiple walls and obstacles, and evaluate on held-out room geometries never seen during training.

Simulation and data. All stages run in Drake. The scene contains a KUKA iiwa arm with a cup end-effector, a ball modeled as a sphere with randomized restitution, and walls modeled as boxes or meshes. Ball positions are observed at a fixed rate with added Gaussian noise of some standard deviation. For each stage, we generate training trajectories by randomizing throw position, velocity, and scene geometry.

Trajectory prediction. We will train a transformer architecture with two kinds of input tokens:

1. Wall tokens: the wall point cloud grouped into local patches, and each patch is embedded with a small PointNet-style encoder [8]
2. Ball tokens: the last k observed ball positions are embedded with their timestamps.

For each time, the model outputs a mean and a per-axis variance of the ball position in the future trajectory. We train it with the Gaussian negative log-likelihood loss function.

Motion planning and control. In general, a catch at some time t requires two conditions:

1. The cup position must coincide with the ball position

2. The cup's opening axis must lie within a cone of some half-angle theta around the direction of the negative velocity vector of the ball.

Each control step solves the differential inverse kinematics as a quadratic program.

Planners. We compare two ways of choosing the catch target:

1. Mean planner: Move the cup to the predicted mean mu(t_c), where the catch time t_c is the earliest time at which the predicted mean is inside the arm's reachable workspace.
2. Uncertainty-aware: we can form some 90% confidence ellipsoid, then move the arm to the configuration that minimizes the worst-case joint-space time to reach any point sampled from the ellipsoid. As the ellipsoid shrinks with new observations, we can begin committing to the mean.

Evaluation

We compare three approaches and an oracle on identical throws (same seeds, rooms, and observation noise) in every stage:

Approach	Predictor	Planner
A: Physics Baseline	Ballistic flight with a mirror reflection off the true geometry, and a fixed nominal restitution	Mean-only
B: Learned, mean-only	Transformer, using the predicted mean only	Mean-only
C: Learned, uncertainty-aware	Transformer, using both the mean and variance predicted	Uncertainty-aware ellipsoid hedging
Oracle	Ground-truth future position	Mean-only

B vs. A measures the value of the learned predictor, C vs. B measures the value of uncertainty information, and the oracle bounds controller-limited performance.

Metrics: We report the following metrics, separately for training and held-out rooms in stage 4:

1. Catch success rate, examined as a function of the number of bounces before catch
2. Prediction mean squared error (MSE)
3. Catch rate vs observation noise
4. Latency per prediction and planning step.

Expected Outcome

Stage 1: We expect near-perfect catching, matching the analytic baseline.

Stages 2–3: We expect prediction error to spike right after each bounce and then recover as the model infers the restitution. The predicted variance should follow the same pattern.

Planner comparison: The uncertainty-aware planner should outperform the mean-only planner mainly in these post-bounce windows and in unseen rooms. There, committing early to a wrong means is costly, and hedging over the uncertainty ellipsoid should reduce this cost. When uncertainty is low, the two planners should perform about the same.

Stage 4: We expect some loss of calibration on held-out rooms.

Latency: With cached wall tokens, we expect inference to stay well under the observation interval, so prediction should not be the bottleneck.

Expected Timeline

Week #	Approx. Dates	Goals
1	10/26–11/1	Drake scene with iiwa, cup, ball, and walls; throw generator and data pipeline; differential-IK catching using ground-truth ball positions
2	11/2–11/8	Transformer and NLL training on stage 1; mean-only planner end-to-end; analytic baseline; **Stage 1 complete**
3	11/9–11/15	Stage 2 single wall data and training; uncertainty-aware planner implemented; first calibration and planner comparison; **Stage 2 complete**
4	11/16–11/22	Stage 3 varied geometries and restitution; point-cloud tokenization implementation; prediction-error and calibration analysis; **Stage 3 complete**
5	11/23–11/29	Stage 4 procedural rooms with held-out test set; **Stage 4 complete**
6	Remaining time	Paper writeup, congregate results and evaluations, finish any leftover tests or ablation studies.

Division of work. JunHui owns the simulation and data generation: the Drake scenes, throw generator, and procedural rooms. Minghao owns the trajectory prediction model: the transformer, training, and calibration and prediction-error analysis. Fumi owns motion planning and control:

differential IK and both catching planners. Integration and the final evaluation runs are shared by everyone.

References

[1] B. Bäuml, T. Wimböck, and G. Hirzinger, "Kinematically optimal catching a flying ball with a hand-arm-system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2010.

[2] G. Olin, L. Chen, N. Gandotra, M. Likhachev, and H. Choset, "Think Fast: Real-Time Kinodynamic Belief-Space Planning for Projectile Interception," Nov. 30, 2025. Accessed: Oct. 23, 2026. [Online]. Available: https://arxiv.org/abs/2512.01108

[3] M. Assran *et al.*, "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," Jun. 11, 2025. Accessed: Oct. 23, 2026. [Online]. Available: https://arxiv.org/abs/2506.09985

[4] M. Parmar, M. Halm, and M. Posa, "Fundamental challenges in deep learning for stiff contact dynamics," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2021.

[5] R. T. Q. Chen, B. Amos, and M. Nickel, "Learning Neural Event Functions for Ordinary Differential Equations," Nov. 08, 2020. Accessed: Oct. 23, 2026. [Online]. Available: https://arxiv.org/abs/2011.03902v4

[6] S. Purushwalkam, A. Gupta, D. M. Kaufman, and B. Russell, "Bounce and Learn: Modeling Scene Dynamics With Real-World Bounces," Apr. 15, 2019. Accessed: Oct. 23, 2026. [Online]. Available: https://arxiv.org/abs/1904.06827v1

[7] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 652–660.

[8] J. J. Park et al., "DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation," CVPR 2019.

Acknowledgements

The author(s) used Claude for finding and understanding related works in the background and formatting references section. It was also used for minor edits of the final proposal after completion.